in general: reward gradient

ideal: $E[\nabla \pi(a|s) R(s,a)]$ → delta

estimate: $\nabla \log \pi(a|s) \delta$: δ is a multiple-choice

check stanford: CS224R-class

⇒ 轨迹概率 = 初始概率 × 策略概率 × 环境概率

def: policy_gradient() def policy_gradient()

avoid sparse/~~bet~~ faster convergence

Def training_walkthrough(): Reward/Advantage/Gradient

GRPO: Group relative policy optimization

⇒ simplification to ppo that removes the critic (value function)

⇒ leverages the group structure in the LM setting (multiple responses per prompt), which provides a natural baseline

// def simple_task():

task: sorting n numbers

prompt: n numbers: prompt = [1, 0, 2]

response: n numbers: response = [0, 1, 2]

Reward should capture how close to sorted the response is

⇒ Define a reward that returns the number of positions where the response matches the ground truth.

e.g. reward = sort_distance_reward([3, 1, 0, 2], [0, 1, 2, 3])

reward = sort_distance_reward([3, 1, 0, 2], [0, 3, 1, 2])

⇒ Define an alternative reward that give more partial credit

e.g. reward = sort_inclusion_ordering_reward([3, 1, 0, 2], [0, 3, 1, 2])